

Documentation technique

HealthAI Coach

MSPR TPRE502 — Bloc E6.2

Équipe : Diana · Anthony · Ilan

1. Introduction

HealthAI Coach est une startup française positionnée sur le marché de la santé connectée et du coaching personnalisé. Dans le cadre de la MSPR TPRES02 (Bloc E6.2), notre équipe a eu pour mission de développer et intégrer des capacités d'intelligence artificielle à la plateforme existante, en répondant aux besoins exprimés par le cahier des charges client.

Cette phase représente une évolution stratégique majeure : transformer une application de suivi de santé en une plateforme de recommandation intelligente, capable d'analyser des photos de repas, de proposer des programmes sportifs personnalisés, et d'adapter ses recommandations aux contraintes médicales et objectifs de chaque utilisateur.

Le document constitue la documentation technique détaillée de l'ensemble des choix architecturaux, algorithmiques et de développement réalisé par l'équipe.

2. Choix des outils utilisés

Nous allons expliquer pourquoi nous avons choisi les outils qui constituent notre stack technique. Chaque choix a été motivé par des contraintes concrètes : coût, performance, confidentialité des données, et adéquation avec l'écosystème existant.

1.1 Back-end Laravel

Laravel a été maintenu comme framework backend principal pour assurer la continuité avec la phase 1 du projet. Pour son écosystème riche et sa documentation extensive en font un choix solide pour une application de santé qui doit gérer des données sensibles.

Packages intégrés durant cette phase :

- Laravel Sanctum : authentification API par tokens sans état (stateless), adapté à une architecture mobile/API où le frontend et le backend sont découplés.
- Lomkit Laravel Rest API : génération automatique d'endpoints Rest standardisés (search, mutate, delete, restore). Ce package évite de répéter du code CRUD pour chaque ressource et impose une structure cohérente à toute l'API.
- Lomkit Access Control : gestion des permissions par périmètre (GlobalPerimeter pour les admins, OwnPerimeter pour les utilisateurs sur leurs propres données).
- Spatie Laravel Permission : système de rôles (admin, coach, user) avec garde API dédié, permettant de différencier les droits selon le profil de l'utilisateur.

- Swagger : génération automatique de la documentation OpenAPI depuis des annotations PHP, garantissant que la documentation reste synchronisée avec le code.

1.2 Micro-service IA – FastAPI (Python)

Nous avons fait le choix de développer le service IA dans un micro-service séparé, conformément aux exigences du cahier des charges qui déclare explicitement que le moteur de recommandation devrait être développé séparément de l'application principale sous forme de micro-service.

FastAPI s'est imposé pour plusieurs raisons :

- Performance : FastAPI est l'un des frameworks Python les plus rapides grâce à son utilisation d'ASGI (Asynchronous Server Gateway Interface). C'est l'essentiel pour un service IA qui peut recevoir des images volumineuses ou devoir répondre rapidement à des requêtes de recommandation
- Typage fort avec Pydantic : la validation automatique des données d'entrée via Pydantic évite les erreurs de type silencieuses et produit des messages d'erreur clairs pour les développeurs frontend.
- Documentation automatique : FastAPI génère nativement une interface Swagger UI sur /docs sans aucune configuration supplémentaire, ce qui facilite les tests et l'intégration
- Ecosystème ML natif : l'intégration avec scikit-learn, joblib, numpy et pymongo est naturelle et bien documentée en Python.

1.3 Modèle de vision – Llava via Ollama

Pour l'analyse de photos de repas, nous avons le choix entre plusieurs solutions : Google Vision API, Hugging Face Transformers, ou des modèles open-source déployés localement.

Nous avons retenu Llava (Large Language and Vision Assistant) déployé via Ollama pour des raisons qui vont au-delà de la simple performance technique.

La principale motivation est la confidentialité des données. Dans un contexte de santé, les photos de repas des utilisateurs sont des données personnelles sensibles. Les envoyer vers les serveurs de Google ou d'OpenAPI pose des questions légitimes de conformité RGPD.

Avec Ollama, le modèle tourne entièrement sur notre infrastructure, sans aucun envoi vers l'extérieur.

L'aspect financier est également important : Google Vision API et les API Hugging Face ont des coûts variables selon le volume. Ollama est gratuit et les coûts sont limités à l'infrastructure d'hébergement.

Enfin, Llava est un modèle multimodal capable d'analyser simultanément une image et du texte pour générer une réponse structurée, exactement ce dont nous avons besoin pour identifier un aliment dans une photo et produire ses valeurs nutritionnelles en JSON.

1.4 Modèle de recommandation – Random Forest (scikit-learn)

Le moteur de recommandation sportive utilise un modèle de machine learning de type Random Forest, entraîné et déployé via scikit-learn. Ce choix mérite une explication, car on aurait pu opter pour un réseau de neurones.

La réalité, c'est que Random Forest est souvent le meilleur choix quand le dataset est de taille modérée. Les réseaux de neurones profonds montrent leur supériorité sur des millions d'exemples, ils tendent à sur-apprendre ou à nécessiter beaucoup de régularisation. Random Forest donne de bons résultats dès que quelques centaines d'exemples.

L'interprétabilité est aussi un facteur clé dans un contexte médical/santé : avec Random Forest, on peut extraire l'importance de chaque variable, ce qui permet d'expliquer à l'utilisateur pourquoi tel exercice lui est recommandé. C'est beaucoup plus difficile avec un réseau de neurones.

Le temps d'inférence est quasi-instantané (quelques millisecondes), ce qui est critique pour une API temps réel. Et joblib permet de sérialiser le modèle entraîné (model.pkl) pour le charger une seule fois au démarrage du serveur, via un singleton.

1.5 Base de données

L'architecture utilise deux bases de données aux rôles complémentaires :

- PostgreSQL (données métier) : la base relationnelle principale, hébergée via Laravel Sail. Elle stocke toutes les données structurées : users, exercices, foods, goals, constraints, health-metrics, subscription, muscle. La nature relationnelle de ces données (un utilisateur a plusieurs contraintes, un exercice appartient à plusieurs catégories) se prête parfaitement au modèle SQL

- MongoDB (historique IA) : les documents produits par les modèles IA ont une structure variable et hétérogène. Une analyse nutritionnelle contient des champs différents d'une recommandation sportive. MongoDB stocke ces documents JSON sans schéma fixe, avec des performances d'écriture très adaptées à un usage de logging.

1.6 Infrastructure Docker

L'ensemble de l'application est conteneurisé. Les deux projets (Backend Laravel et API IA FastAPI) sont définis dans des fichiers docker-compose.yml distincts, correspondant à deux dépôts git séparés et deux périmètres d'équipes différents.

Pour permettre la communication, nous avons mis en place un réseau Docker externe partagé nommé healthai_backend_sail. Ce réseau est créé par le projet Backend (Laravel Sail) et rejoint par le projet API IA via external : true dans son docker-compose.yml. Tous les containers (FastAPI, Ollama, MongoDB) peuvent aussi communiquer avec Laravel par nom des container (http://healthai_fastapi:4000), sans dépendance à des adresses IP dynamiques qui changeraient à chaque redémarrage.

1.7 Frontend React et Chart.js

Le frontend a été développé en React, choisi après comparaison avec Vue.js et Angular. React s'impose pour trois raisons principales : son modèle de composants réutilisables correspond parfaitement à une interface de coaching (carte d'exercice, carte nutritionnelle, widget de métriques — tous réutilisables), son écosystème est le plus riche du marché (hooks, context API, bibliothèques de visualisation), et sa courbe d'apprentissage progressive permettait à l'équipe de monter en compétence rapidement.

Vue.js aurait été une alternative viable pour sa simplicité, mais son écosystème de visualisation de données est moins mature. Angular a été écarté car sa complexité architecturale (modules, decorators, injection de dépendances obligatoire) était disproportionnée pour les besoins du projet.

Pour les visualisations de données (courbes d'évolution du poids, graphiques de progression sportive, répartition nutritionnelle), Chart.js a été retenu face à D3.js et Plotly.js. Chart.js offre un excellent rapport complexité/résultat — ses graphiques standard (lineChart, barChart, doughnut) couvrent tous nos besoins sans nécessiter

la courbe d'apprentissage très steep de D3.js. D3.js aurait été pertinent pour des visualisations sur-mesure complexes, ce qui n'est pas notre cas.

3. Choix des algorithmes et APIs utilisés

3.1 Analyse nutritionnelle – Llava et architecture du prompt

L'analyse de photos de repas repose sur Llava 7B, un modèle de vision multimodal open-source. Son utilisation implique de construire un prompt soigneusement conçu pour obtenir une réponse structurée et exploitable.

Architecture de prompt :

Le prompt suit une structure en trois parties que nous appelons « chain-of-thought guidé » :

- Rôle et contexte « You are a nutrition expert. Analyze the meal and respond with ONLY valid JSON ». Définir le rôle force le modèle à adopter une posture d'expert plutôt que d'écrire l'image de manière générique.
- Règles d'analyse (ANALYSIS APPROACH) : guide explicite demandant au modèle d'identifier d'abord si l'image montre un ou plusieurs aliments, d'observer les détails visuels (texture des graines, couleur, forme), puis de nommer précisément avant de calculer. Cette étape a été ajoutée après avoir observé que Llava confondait un fruit de la passion coupé avec une salade de fruits, erreur due à l'absence de processus d'observation structuré.
- Structure JSON exacte attendue : spécifier le format de sortie avec des exemples de valeurs et des enums valides (cooking_method, meal_type) réduit considérablement le taux de réponses malformées.

La température du modèle est fixée 0.1 (très basse) pour privilégier la cohérence et la répétabilité des analyses plutôt que la créativité.

Limitations connues

Llava 7B montre des limites sur la reconnaissance fine d'aliments peu représentés dans ses données d'entraînement. Des erreurs de classification (fruit exotique confondu avec une préparation plus commune) ont été observées. Ces limitations sont inhérentes à la taille du modèle et à son entraînement généraliste. Une perspective d'évolution serait l'utilisation d'un modèle spécialisé en reconnaissance alimentaire (fine-tuné sur un dataset comme Food-101) ou d'une version 13B de Llava.

3.2 Recommandation sportive — Random forest

Feature d'entrée et encodage

Le modèle Random forest utilise 4 variables d'entrées, toutes converties en valeurs numérique avant passage au modèle :

Variable	Type en entrée	Encodage numérique
physical_activity_level	String (sedentary/moderate/active)	0 / 1 / 2
bmi	Float (poids/taille ²)	Valeur décimale directe
age	Calculé depuis birthdate	Entier (années)
favorite_exercise_category	String (Cardio/Musculation/Poids du corps)	0 / 1 / 2

Pipeline d'entraînement

Le pipeline suit plusieurs étapes structurées. Le nettoyage des données par supprimer les valeurs impossibles (BMI négatif ou supérieur à 70, valeurs aberrantes biométriques) et les exercices rares comptant moins de 20 références dans le dataset, un exercice trop peu représenté ne peut pas être appris correctement par le modèle. Les textes sont ensuite convertis en valeurs numérique via encodage.

La mise en forme des données utilise un Scaler pour normaliser toutes les valeurs numériques dans un intervalle 0-1. Cette étape est indispensable car les features ont des plages très différents (BMI entre 15 et 40, âge entre 15 et 80) sans normalisation, les variables avec les plus grandes valeurs domineraient artificiellement le modèle.

Choix du modèle

Deux familles de modèles ont été testées en comparaison. Le modèle linéaire est plus rapide et efficace si le problème est linéairement séparable. L'ensemble d'arbres de décision (Random forest) est plus précis et robuste, c'est ce dernier qui a été retenu, car la relation entre le profil utilisateur et les exercices recommandés n'est pas linéaire.

L'approche de types de classification a été choisie : l'objectif est de trouver, parmi tous les exercices disponibles, ceux qui correspondent le mieux au profil de l'utilisateur. Une tentative de modèle multi-label a également été explorée pour permettre de recommander plusieurs exercices simultanément, mais le modèle single-label s'est révélé plus stable.

Métrique d'évaluation

Quatre métriques ont été utilisées pour évaluer les performances du modèle. La Top-k Accuracy vérifie si la bonne réponse se trouve dans les 5 premières prédictions métrique la plus pertinente car l'application retourne les 5 meilleurs exercices. La First good answer indique, parmi les prédictions triées par probabilité décroissante, à quelle position apparaît la réponse attendue. La Learning Curve compare les performances sur les données d'entraînement vs les données de test pour détecter les sur-apprentissage (overfitting). La feature importance révèle quelle variable a le plus d'impact sur la recommandation.

Stockage des résultats

Lors de chaque appel API, le modèle score les 250 exercices et les retourne triés par probabilité décroissante. Les 10 meilleurs sont enregistrés dans MongoDB pour constituer un historique des recommandations, permettant à terme d'analyser les tendances et d'affiner le modèle.

3.3 Filtrage métier post-IA – IllegalExercisesHandler

Le Random forest est un modèle statique : il ne connaît ni les blessures de l'utilisateur, ni ses objectifs personnels. C'est pourquoi nous avons développé une couche de filtrage métier qui s'applique après la prédiction IA, avant de retourner les résultats au client.

- Règle de pertinence (goal) : un exercice est exclu si aucun de ses objectifs associés ne correspond aux objectifs définis par l'utilisateur. Par exemple, recommander un exercice de « prise de masse » à quelqu'un dont l'objectif est la « perte de poids » n'aurait aucun sens. Si l'utilisateur n'a aucun objectif défini, cette règle est neutralisée pour éviter d'exclure tous les exercices.
- Règle de sécurité (contrainte médicale) : un exercice est exclu s'il partage au moins une contrainte médicale avec l'utilisateur. Un exercice contre-indiqué pour les blessures au genou ne sera pas proposé à un utilisateur souffrant de cette condition. La comparaison utilise l'intersection des identifiants de contraintes entre l'utilisateur et l'exercice.

Les deux règles utilisent l'opération d'intersection (`Collection::intersect()` en Laravel) sur les identifiants UUID, une approche simple, efficace, et facilement testable.

Un comportement fail-safe a été implémenté : si un exercice recommandé par le Random forest n'existe pas dans la base de données PostgreSQL (les nomenclatures ne sont pas encore parfaitement alignées entre le dataset ML et la base applicative), il est conservé par défaut plutôt qu'exclu. Ce choix garantit que l'application reste fonctionnelle et présente des résultats même en l'absence d'un alignement complet des données.

4. Architecture technique

4.1 Vue d'ensemble

L'architecture de HealthAI Coach repose sur séparation claire entre le backend métier (Laravel) et le service IA (FastAPI), communiquant via un réseau Docker partagé. Le SDK PHP `mspr2/sdk-ia` encapsule toute la communication entre ces deux composants.

Le flux d'une requête de recommandation suit le chemin suivant : l'utilisateur envoie une requête au backend Laravel (authentifié via Sanctum), qui délègue au SDK PHP, lequel appelle le service FastAPI via Guzzle HTTP. FastAPI charge les modèles ML et retourne les prédictions. Laravel filtre ensuite les résultats via `IllegalExercisesHandler` avant de répondre au client. En cas d'indisponibilité de FastAPI, le SDK retourne automatiquement un statut « degraded » sans propager d'erreur 500.

4.2 SDK PHP mspr2/sdk-ia

Le SDK est un package Composer local structuré en couches distinctes avec des responsabilités séparées. Les clients (OllamaClient, RecommendationClient) gèrent les appels http Guzzle et expose une Facade Laravel. IAManager orchestre ses clients via injection de dépendances et expose une Facade Laravel pour un accès simplifié depuis les controllers.

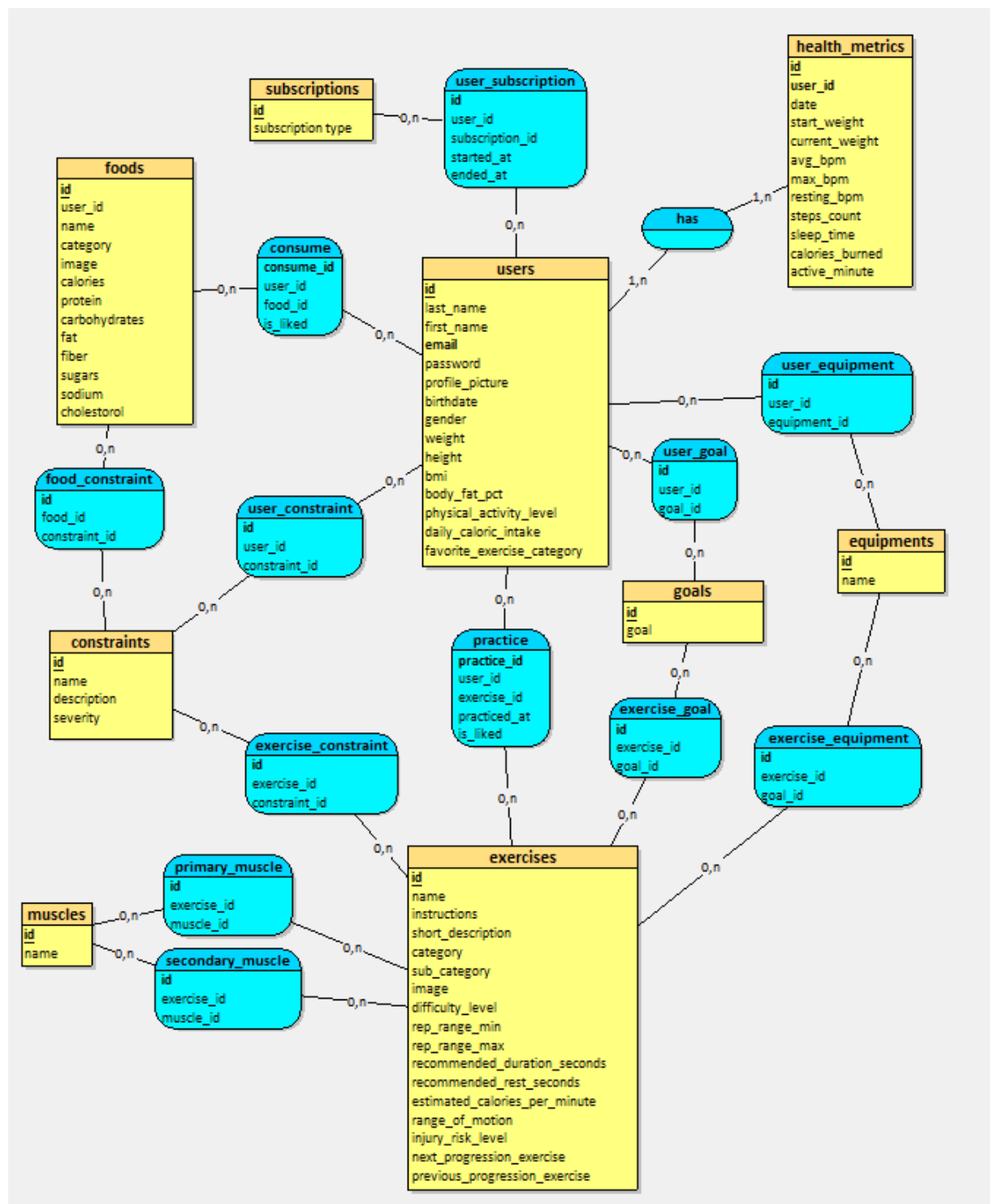
IllegalExercisesHandler applique les règles métier post-IA. SDKIAServiceProvider enregistre le tout comme singleton dans le conteneur IoC de Laravel.

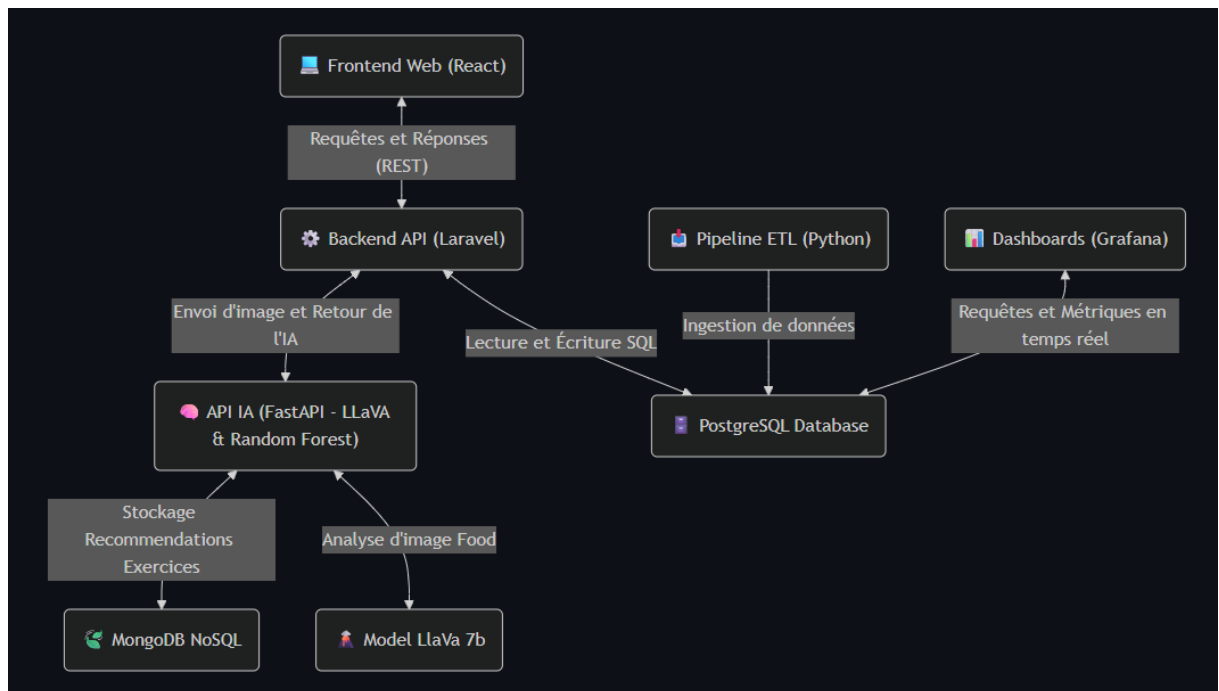
Cette architecture en couches permet de tester chaque composant indépendamment et de faire évoluer, par exemple, le protocole de communication (HTTP vers GRPC) sans toucher à la logique métier.

5. Modèle de données

Plusieurs adaptations ont été apportées au modèle de données existant pour supporter les nouvelles fonctionnalités IA :

- Colonne favorite_exercise_category sur users : alimente directement le modèle Random forest. Peut être mise à jour manuellement par l'utilisateur ou automatiquement via l'historique de pratique (perspective d'évolution)
- Table pivot user_goal, user_constraint, exercise_goal, exercise_constraint : permettant au filtre IllegalExercisesHandler de croiser les données de l'utilisateur avec celles de chaque exercice recommandé
- Table health_metrics : colonne weight ajoutée pour la pesée de l'utilisateur. Cette valeur déclenche automatiquement la mise à jour du profil via un Event/Listener.





6. Événements et mise à jour automatique du profil

Lorsqu'un utilisateur enregistre une nouvelle pesée (POST /health-metrics/mutate). Le event dans EventServiceProvider et le Listener UpdateUserWeight réagit à cet événement en mettant à jour le health metrics le poids de l'utilisateur et en recalculant son IMC selon la formule internationale : $BMI = \text{poids_kg} / (\text{taille_m})^2$.

Ce pattern Event/Listener a été préféré à un Observer pour sa meilleure extensibilité : ajouter une notification push ou un calcul de tendance sur la pesée ne nécessitera que la création d'un nouveau Listener, sans toucher au code existant. C'est une application directe du principe Open/Closed (ouvert à l'extension, fermé à la modification).

Par ailleurs, lors de création d'un utilisateur via l'API, la méthode mutated() de UserResource calcule automatiquement le BMI et l'apport calorique journalier via la formule de Harris-Benedict révisée, qui tient compte du poids, de la taille, de l'âge, du genre et du niveau d'activité physique.

7. Difficultés rencontrées

Cette section documente les principaux obstacles techniques rencontrés durant le développement. Ces éléments constituent une trace utile pour toute équipe reprenant le projet.

Diana :

Laravel IA et FastAPI Ollama :

- Communication inter-containers Docker entre 2 projets distincts, chaque docker-compose créer son propre réseau isolé, réseau docker externe partagé healthai_backend_sail avec external : true dans le projet API IA.
- Llava 7B confond fruit de la passion avec salade de fruits : absence de processus d'analyse structuré dans le prompt, amélioration du prompt avec analysis approach et exemple concrets, limitation documentée.

Anthony :

Recommandation sportive et FastAPI IA

- Lenteur de programmes
- Mauvaise direction
- Manque de métrique approprié
- Manque de réflexion sur les données disponible

Ilan :

Frontend et infrastructure Docker :

- Manque de temps
- Accessibilité
- Responsive
- Automatisation Stack

8. **Principes d'ergonomie et normes d'accessibilité**

Bien que notre périmètre soit principalement backend, les d'API que nous avons faits ont un impact direct sur l'ergonomie et l'accessibilité de la future interface utilisateur.

8.1 **Design de l'API orienté utilisateur**

- Messages d'erreur explicites : toutes les réponses d'erreur incluent un champ message lisible par l'humain et un champ errors détaillant chaque champ invalide. Le

frontend peut ainsi afficher des messages d'aide précis à l'utilisateur plutôt que des codes d'erreur opaques

- Mode dégradé documenté : quand le service IA est indisponible, l'API retourne un statut « degraded » avec `is_working: 0` plutôt qu'une erreur 500. L'interface peut détecter ce statut et proposer une saisie manuelle, évitant une expérience bloquante pour l'utilisateur
- Champs optionnels avec fallback intelligent : `favorite_exercise_category` est optionnel dans `/ai/recommend` — si absent, la valeur du profil est utilisée. Si le profil n'en a pas, un défaut raisonnable est appliqué. Cela réduit la friction pour l'utilisateur
- Normalisation insensible à la casse : les catégories d'exercice sont normalisées côté serveur (`cardio` → `Cardio`), évitant des erreurs de validation frustrantes dues à une simple différence de casse

8.2 Principes d'ergonomie et normes d'accessibilité

La documentation API est générée automatiquement via L5 Swagger depuis des annotations PHP directement dans le code. Chaque endpoint est documenté avec ses paramètres, les exemples de body, et les différents cas de réponse (succès, mode dégradé, erreurs de validation). Cette documentation interactive permet aux développeurs frontend de tester les endpoints sans outil externe, accélérant l'intégration et réduisant les allers-retours

8.3 Considérations d'accessibilité pour l'interface (WCAG/RGAA)

Les recommandations suivantes ont été formulées à destination du développeur frontend, en cohérence avec les exigences WCAG niveau AA mentionnées dans le cahier des charges :

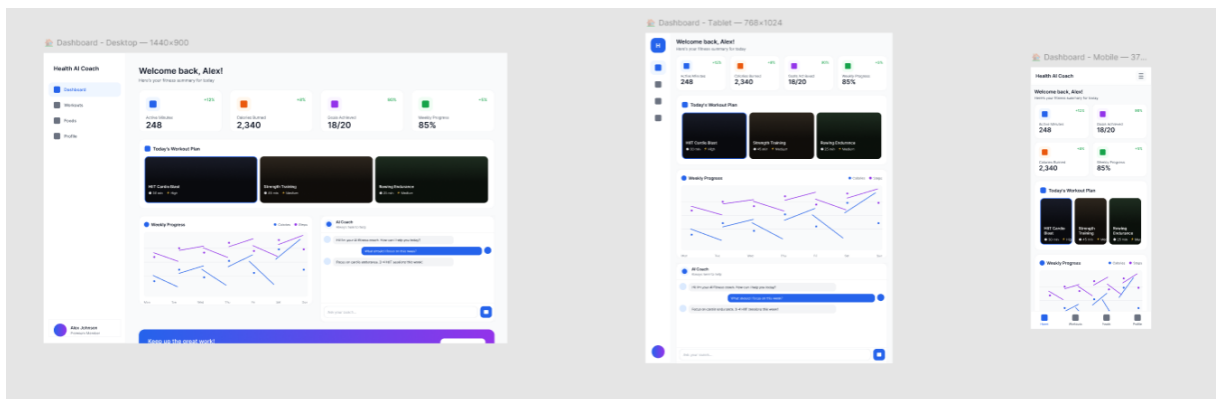
- Indicateur de confiance IA : le score de confiance retourné par LLaVA (0-100) doit être affiché de manière accessible mais aussi en texte (ex: « Confiance : 85% ») ou via une jauge avec `aria-label`
- Recommandations d'exercices : chaque exercice recommandé doit inclure une description textuelle accessible, pas seulement un nom et un score
- Navigation clavier : les formulaires de soumission d'image et de sélection de catégorie d'exercice doivent être entièrement navigables au clavier
- Contraste des couleurs : le statut « degraded » ne doit pas être signalé uniquement par la couleur rouge — utiliser une icône ou un texte explicatif en complément

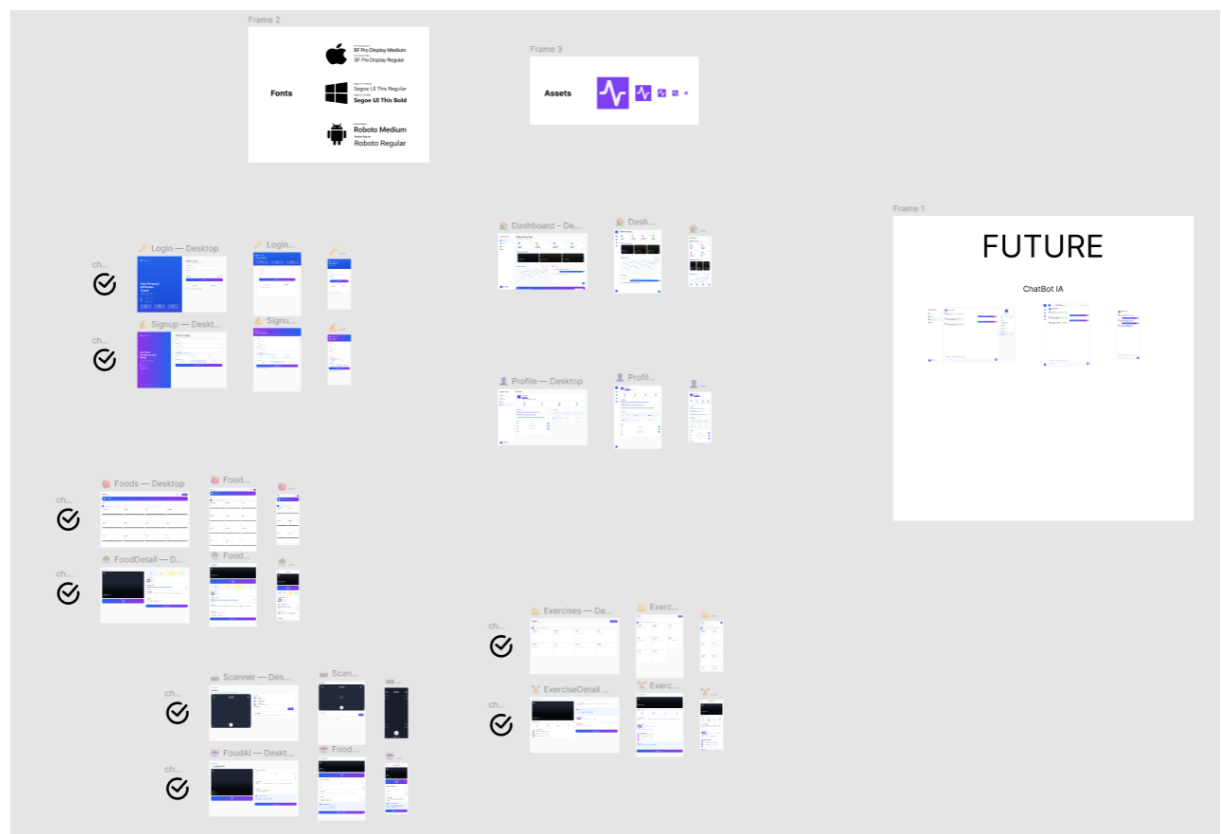
8.4 Maquettes et conception UX — Figma

La conception de l'interface a débuté par une phase de maquettage sur Figma, permettant de valider l'ergonomie et l'organisation de l'information avant toute ligne de code. Figma a été choisi pour sa collaboration temps réel (toute l'équipe pouvait commenter et itérer sur les maquettes simultanément) et sa capacité à exporter directement les spécifications CSS vers les développeurs.

Les maquettes couvrent trois formats pour assurer le responsive design : mobile (375px), tablette (768px) et desktop (1280px).

Les principes d'ergonomie appliqués incluent une navigation principale en barre inférieure sur mobile, un parcours utilisateur en 3 étapes maximum pour accéder à une recommandation, et des états de chargement explicites pour les appels IA qui peuvent prendre plusieurs secondes.





9. Perspectives d'évolution

Cette section identifie les axes d'amélioration prioritaires identifiés durant le développement.

- Alternative textuelle pour l'analyse de repas : si l'analyse échoue (mode dégradé) , proposer une saisie manuelle des aliments avec des champs bien étiquetés (label explicite, pas seulement placeholder)
- Navigation clavier : les formulaires de soumission d'image et de sélection de catégorie d'exercice doivent être entièrement navigables au clavier
- Modèle LLaVA 13B : améliorer la précision de reconnaissance des aliments en passant à la version 13B, avec évaluation sur un dataset d'aliments exotiques
- Modulation du déficit calorique selon le goal : adapter le calcul Harris-Benedict en fonction de l'objectif (déficit -400 kcal pour perte de poids, surplus +300 kcal pour prise de masse, TDEE brut pour maintien)
- Endpoint de planification nutritionnelle : développer un endpoint /ai/meal-plan qui génère un plan de repas journalier personnalisé basé sur l'apport calorique calculé par Harris-Benedict, en utilisant un modèle LLM texte (Llama 3) plutôt que LLaVA
- Mise à jour automatique de favorite_exercise_category : recalculer automatiquement la catégorie préférée de l'utilisateur après chaque séance enregistrée, basé sur la catégorie la plus pratiquée dans les 30 derniers jours

Conclusion

En 19 heures conclusion de développement, notre équipe a réussi à intégrer des capacités d'intelligence artificielle fonctionnelles à la plateforme HealthAI Coach. Nous avons développé un SDK PHP robuste, un micro-service IA en FastAPI, un pipeline d'analyse d'images via Llava, un moteur de recommandation basé sur Random forest, est une couche de filtrage métier adaptée aux contraintes médicales des utilisateurs.

Les difficultés rencontrées, communication inter-containers Docker, mapping des données entre systèmes hétérogènes, alignement des nomenclatures ML et base de données, nous ont poussés à concevoir des solutions robustes plutôt que des contournements rapides. Le mode dégradé systématique, les comportements fail-safe, et la documentation des limitations connues reflètent une approche industrielle sérieuse.

Les perspectives d'évolution constituent une feuille de route claire pour les prochaines phases du projet, avec en priorité l'alignement des nomenclatures entre le dataset ML et la base applicative, qui permettra d'activer pleinement le filtrage métier sur les recommandations IA.